Calcetto Web App - Architettura, Algoritmi e Guida allo Sviluppo

Sistema Multi-Piattaforma PWA con Motore Elo e Sincronizzazione Cloud Supabase

Alessandro Harej
Politecnico di Milano

18 ottobre 2026

Sommario

Questo documento tecnico descrive l'architettura, i principi matematici, la struttura dei dati e le procedure operative alla base della web application **Calcetto** sviluppata per la gestione delle leghe di calcio balilla. Viene documentata la migrazione completa dall'originaria implementazione monolitica in Python/Streamlit a un'architettura disaccoppiata client-serverless basata su Progressive Web Application (PWA) statica (ospitata gratuitamente su GitHub Pages) e un database cloud PostgreSQL relazionale (gestito tramite Supabase). Vengono inoltre dettagliate le formulazioni dell'algoritmo di matchmaking combinatorio, il sistema di rating Elo con coefficiente di volatilità $K = 40$, la gestione multi-gruppo (multi-tenant) e le istruzioni operative per manutenzioni e aggiornamenti futuri.

Indice

1 Introduzione e Diagnosi del Sistema Legacy **2**
1.1 L'Implementazione Streamlit Originale . . . 2
1.2 Criticità Riscontrate . . . 2

2 Nuova Architettura Serverless & Mobile-First **2**

3 Modello Matematico e Algoritmi **3**
3.1 Sistema di Rating Elo per Squadre 2v2 . . . 3
3.1.1 Fattore di Sviluppo $K = 40$ . . . 3
3.2 Algoritmo di Matchmaking e Generazione Equa . . . 4

4 Architettura Multi-Gruppo (Versione V2) **4**

5 Strumenti di Amministrazione da PC **4**

6 Automazione Keepalive (GitHub Actions) **5**

7 Guida Operativa per Sviluppi Futuri **5**
7.1 Struttura delle Cartelle . . . 5
7.2 Best Practices per le Modifiche al Codice . . . 5

1 Introduzione e Diagnosi del Sistema Legacy

1.1 L'Implementazione Streamlit Originale

L'applicazione era inizialmente concepita come uno script Python monolitico (`app.py`) eseguito su Streamlit Community Cloud. La persistenza dei dati avveniva attraverso un file JSON (`statistiche.json`) salvato direttamente nel repository GitHub. A ogni conclusione di partita o modifica dei presenti:

1. L'applicazione serializzava l'intero dizionario delle statistiche in formato JSON.
2. Tramite la libreria `PyGithub` e un Personal Access Token (PAT), veniva effettuato un commit Git programmatico direttamente sul branch `main` del repository.

1.2 Criticità Riscontrate

- **Continuous Deployment Loop:** Streamlit Community Cloud monitora i commit sul repository di riferimento. Ogni commit inviato da `PyGithub` innescava automaticamente il riavvio del container e la ricompilazione dello script.
- **Azzeramento dello Stato (`st.session_state`):** Il riavvio del container causava la perdita istantanea delle variabili in memoria, costringendo i giocatori sul campo a ricaricare la pagina e perdendo la partita corrente in corso.
- **Concorrenza e Race Conditions:** Qualora due utenti inviassero contemporaneamente i risultati o modificassero le presenze, Git sollevava eccezioni di conflitto di merge (*Fast-Forward Conflict*), causando il crash dell'app.
- **Tempi di Avvio e Cold Starts:** L'avvio dell'istanza Python richiedeva diversi secondi (spesso oltre 10-15s), rendendo l'esperienza utente da smartphone poco adatta a un uso rapido in pausa pranzo.

2 Nuova Architettura Serverless & Mobile-First

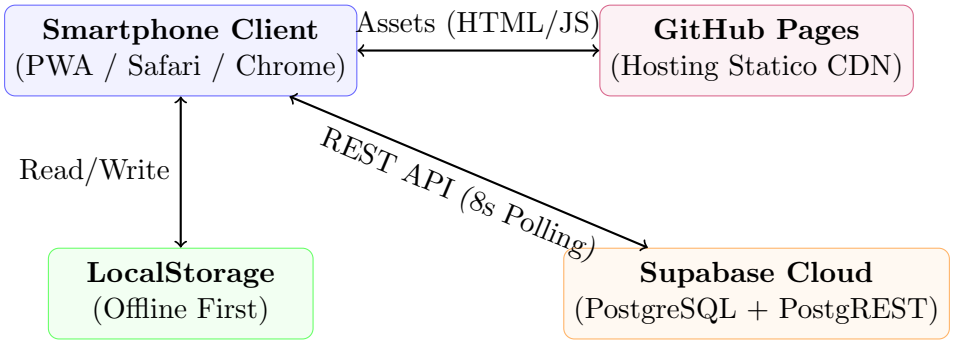


Figura 1: Diagramma del flusso architetturale della nuova applicazione.

L'architettura è stata rifattorizzata separando nettamente il livello di presentazione dal livello di persistenza:

1. **Frontend (Client-side):** Realizzato in HTML5 semantico, TailwindCSS (tramite CDN per una grafica scura, moderna e reattiva su schermi di qualsiasi dimensione) e Vanilla JavaScript (`app.js`). Non richiede build pipeline complesse (né Node.js, né Webpack).

2. **Hosting Statico (GitHub Pages):** Gratuito, distribuito su CDN globale con certificato SSL automatico e tempi di caricamento istantanei (< 200 ms).

3. **Storage e Database (Supabase Cloud):** Database PostgreSQL con API RESTful istantanee generate da PostgREST. La sincronizzazione avviene tramite protocollo HTTP/JSON mediante `fetch` asincrono.

4. **Offline-First & PWA Support:** Un Service Worker (`sw.js`) memorizza nella cache del browser i componenti dell'interfaccia. Tramite il file `manifest.json`, l'app può essere installata sulla schermata home dello smartphone come fosse un'applicazione nativa senza barre del browser.

3 Modello Matematico e Algoritmi

3.1 Sistema di Rating Elo per Squadre 2v2

Il sistema adotta l'algoritmo Elo standard applicato alle competizioni a coppie, calcolando il punteggio medio di ciascuna formazione.

Dati due giocatori della Squadra A con rating R_{A1} e R_{A2}, e due della Squadra B con rating R_{B1} e R_{B2}, i rating aggregati sono definiti come:

$$R_A = \frac{R_{A1} + R_{A2}}{2}, \qquad R_B = \frac{R_{B1} + R_{B2}}{2} \tag{1}$$

La probabilità attesa di vittoria della Squadra A (E_A) e della Squadra B (E_B) è calcolata secondo la funzione logistica standard:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, \qquad E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}} = 1 - E_A \tag{2}$$

3.1.1 Fattore di Sviluppo $K = 40$

Al termine del match, indicando con $S_A \in \{1, 0\}$ l'esito per la Squadra A (1 per vittoria, 0 per sconfitta):

$$\Delta R_A = K \cdot (S_A - E_A) \tag{3}$$

$$R'_{A1} = R_{A1} + \Delta R_A, \qquad R'_{A2} = R_{A2} + \Delta R_A \tag{4}$$

Significato del valore $K = 40$: Nelle federazioni scacchistiche (FIDE), K varia tipicamente tra 10 e 20 per giocatori stabili, e 40 solo per principianti. Nel contesto del calcetto universitario e aziendale:

- Il numero totale di partite giocate in un anno è ridotto (spesso 1-3 partite al giorno per giocatore).
- Con un K basso (es. 15), battere i campioni del torneo produrrebbe un avanzamento trascurabile (+3 o +5 punti), rendendo la classifica statica e noiosa.
- Con $K = 40$, una vittoria contro una squadra nettamente favorita assegna circa +30/+35 punti, mentre una vittoria contro sfavoriti assegna circa +8/+12 punti. Ciò garantisce dinamismo, competizione e ricompense percepibili partita per partita.

3.2 Algoritmo di Matchmaking e Generazione Equa

Per garantire rotazione equilibrata e formazioni avvincenti:

1. **Priorità di Partecipazione:** I presenti vengono ordinati in base al numero di partite giocate nella sessione corrente (`partite_giocate_oggi`). I 4 colleghi con il minor numero di partite vengono scelti prioritariamente.
2. **Valutazione delle Combinazioni (se Ranking attivo):** Dati i 4 candidati scelti $\{P_0, P_1, P_2, P_3\}$, esistono esattamente tre possibili accoppiamenti 2v2:

$$C_1 : [P_0, P_1] \text{ vs } [P_2, P_3], \quad C_2 : [P_0, P_2] \text{ vs } [P_1, P_3], \quad C_3 : [P_0, P_3] \text{ vs } [P_1, P_2]$$

   L'algoritmo valuta una funzione di penalità $J(C_k)$ per ciascuna configurazione:

$$J(C_k) = |R_{\text{sq1}} - R_{\text{sq2}}| + \text{Pen}_{\text{ruoli}} + \text{Pen}_{\text{ripetizione}} \tag{5}$$

   dove:

   - $\text{Pen}_{\text{ruoli}} = +500$ se una squadra ha 2 Difensori puri, $+150$ se ha 2 Attaccanti puri.
   - $\text{Pen}_{\text{ripetizione}} = +5000$ se la coppia ha già giocato insieme nella stessa giornata.

   Viene selezionata la combinazione che minimizza $J(C_k)$.

4 Architettura Multi-Gruppo (Versione V2)

Per consentire l'utilizzo dell'applicazione a più gruppi di ricerca, dipartimenti o uffici diversi senza condividere o sovrascrivere le statistiche:

- **Struttura Tabella Supabase (`calcetto_state`):**

Colonna	Tipo	Descrizione
`id`	`text (PK)`	Identificativo gruppo (es. `global`, `bioeng`)
`data`	`jsonb`	Stato completo (giocatori, presenze, match, PIN)
`updated_at`	`timestamptz`	Timestamp dell'ultima modifica (gestione concorrenza)

- **Routing via Query String:** L'app in `/v2/` legge il parametro URL:

  `https://harejal.github.io/Calcetto/v2/?group=bioeng`

  Tutte le letture e le scritture REST caricano esclusivamente la riga avente `id = 'bioeng'`.

- **PIN di Amministrazione:** Ciascun gruppo definisce il proprio PIN (memorizzato in `data.admin_pin`). Le azioni distruttive (eliminazione giocatori, reset classifica stagionale) sono protette e richiedono l'inserimento del PIN tramite modal personalizzata su mobile.

5 Strumenti di Amministrazione da PC

Nel repository sono inclusi due script Python da riga di comando per il super-amministratore (eseguibili tramite l'interprete Anaconda presente sul PC):

1. `resetClassifica.py`: Script che si collega a Supabase e azzera l'Elo stagionale riportando tutti a 1200 punti senza eliminare i nominativi e i ruoli.
2. `gestioneTeam.py` (e il relativo lanciatore `gestisci_team.bat`): Script interattivo che elenca tutti i team registrati nel Cloud, permette di ispezionare le singole classifiche, azzerare singoli team o cancellare definitivamente un team dal database.

6 Automazione Keepalive (GitHub Actions)

I piani gratuiti di Supabase mettono in pausa automatica i database inattivi per più di 7 giorni consecutivi. Per garantire continuità durante le vacanze o i periodi di esami, è stato implementato il workflow:

`.github/workflows/keepalive.yml`

Il workflow esegue un `curl` con query minima (`limit=1`) ogni 3 giorni alle 08:00 UTC, mantenendo il database costantemente attivo senza alcun intervento manuale.

7 Guida Operativa per Sviluppi Futuri

7.1 Struttura delle Cartelle

- `index.html`, `app.js`, `sw.js`, `manifest.json`: Versione di produzione (gruppo PHD principale).
- `v2/`: Versione multi-gruppo per gli altri team universitari.
- `gestioneTeam.py`, `resetClassifica.py`: Strumenti di gestione lato amministratore.
- `.github/workflows/`: Automazioni CI/CD e keepalive.

7.2 Best Practices per le Modifiche al Codice

1. **Invalida la Cache del Service Worker:** Se modifichi `app.js` o l'interfaccia HTML, incrementa la versione della stringa `CACHE_NAME` in `sw.js` (es. da `calcetto-v1.3` a `calcetto-v1.4`). In questo modo, gli smartphone degli utenti scaricheranno automaticamente l'aggiornamento alla prima apertura.
2. **Preserva la Struttura JSON:** Quando aggiungi nuovi campi all'oggetto `state`, assicurati di impostare valori di default nel caso in cui un utente abbia in cache una versione precedente del JSON locale.